I have analyzed the **MicroLend** core monolith backend architecture provided. I will maintain the full context of these specific source directories and endpoints for all our future conversations whenever you reference microlend src.
Here is the step-by-step overview and mapping of the system's APIs structured exactly in your specified business workflow order, detailing the endpoint paths, HTTP verbs, and responsible roles.

Workflow Overview

The application uses a **Modular Monolith** pattern inside a single Spring Boot process. Security is managed through standard JWT tokens with specific role authorization prefixes (e.g., /api/admin/**, /api/field-officer/**, /api/borrower/**) mapping onto the 6 custom roles: NBFC_ADMIN, FIELD_OFFICER, CREDIT_OFFICER, BRANCH_MANAGER, COLLECTIONS_OFFICER, and BORROWER.

API Workflow Sequence Mapping

Step	User Role	Business Action	HTTP Method & API Endpoint	Internal Controller / Mechanics
1	Admin / User	**Login**	POST /api/auth/login	AuthController -> Generates token
2	Any User	**Reset Password** (On first login/voluntary)	POST /api/auth/reset-password	AuthController -> Updates hash via BCrypt
3	NBFC Admin	**Create Loan Product**	POST /api/admin/loan-products	LoanProductAdminController
4	Field Officer	**Login**	POST /api/auth/login	Logs in and gets full session token
5a	Field Officer	**Creates Center**	POST /api/field-officer/centres	FieldOfficerOriginationController

5b	Field Officer	**Creates Group**	POST /api/field-officer/groups	FieldOfficerOriginationController
6	Field Officer	**Creates/Registers Borrower**	POST /api/field-officer/borrowers	FieldOfficerBorrowerController (auto-provisions user account)
7	Field Officer	**Upload KYC File**	POST /api/field-officer/borrowers/{borrowerId}/kyc	FieldOfficerBorrowerController (Multipart request)
8	Credit Officer	**Login & View Verification File**	GET /api/credit-officer/kyc/{kycId}/file	CreditOfficerKycController (inline stream)
9	Credit Officer	**Approve KYC Status**	PUT /api/credit-officer/kyc/{kycId}/verify	CreditOfficerKycController
10	Field Officer	**Create Loan Application**	POST /api/loan-applications	LoanApplicationController (Fires credit scoring engine instantly)
11	Credit Officer	**Review Assessment & Decide Application**	PUT /api/credit-officer/applications/{id}/decision	CreditOfficerApplicationController -> Auto-generates Sanction Letter if

				action=APPROVE
12	Borrower	**Login & View Sanction Letters**	GET /api/borrower/sanction-letters	BorrowerSanctionController
13	Borrower	**Accept Sanction Letter**	PUT /api/borrower/sanction-letters/{id}/accept	BorrowerSanctionController -> Creates LoanAccount & Amortization Schedule automatically
14	Field Officer	**Collect Monthly EMI**	POST /api/field-officer/collections	FieldOfficerCollectionController (Creates **Pending** receipt state)
15	Borrower	**Accept Digital Script / Receipt**	PUT /api/borrower/receipts/{id}/approve	BorrowerReceiptController -> Flips status to **Paid** and reduces outstanding principal
16	Branch Manager	**Portfolio Analysis (PAR & NPA)**	GET /api/analytics/par-distribution GET /api/analytics/npa-trend	AnalyticsController (Computes live counts for PAR 30/60/90 from trans. tables)
17	System Task	**Automatic Overdue**	POST /api/admin/delinquency/run	Scheduled via DelinquencyScheduler or

		Processing		manual trigger via AdminDelinquencyController
18	Branch Manager	**Assign Delinquent Case**	PUT /api/branch-manager/delinquency-cases/{id}/assign	BranchManagerDelinquencyController -> Assigns a collection officer
19	Collections Ofc	**Update Schedule / Management Fallback**	PUT /api/branch-manager/receipts/{id}/co-sign	BranchManagerReceiptController (Allows Manager or Collections Officer to manually update schedule via fallback co-sign)

Implementation Instructions

All these endpoints run out of the unified microlend-core monolithic instance configured on server port 8090.

- **Context tracking:** The system implements a custom CorrelationIdFilter which sets up an X-Correlation-Id tracking token on execution paths. Ensure this is tracked in your logs across these microfinance steps.
- **Security Context Enforcement:** Methods implicitly pull identification parameters such as registeredByFieldOfficerId and currentUserId server-side out of the context parameters mapping into SecurityUtil.currentUserId(). You do not need to pass user identity IDs inside the JSON bodies for creation steps.